LakeRun

A local CDC lakehouse: Postgres to Kafka to Iceberg, with DuckDB and bronze, silver and gold layers

Concepts, end-to-end flow, design decisions, how it runs in a real product, and interview questions with answers.

Built and verified on the local stack: Postgres 16, Debezium 2.7, Kafka (KRaft), MinIO, Iceberg REST catalog, DuckDB 2.0.
November 2026

Contents

1. The 60-second pitch
2. Architecture at a glance
3. Core concepts, from source to dashboard
4. Life of one order: the flow end to end
5. The medallion layers in this project
6. Design decisions and trade-offs
7. What was broken, and how it was fixed
8. How this works in a real product
9. Interview questions and answers
10. Hands-on: commands and exercises
11. Load test: 1 million events, before and after scaling
12. Limitations and next steps

1. The 60-second pitch

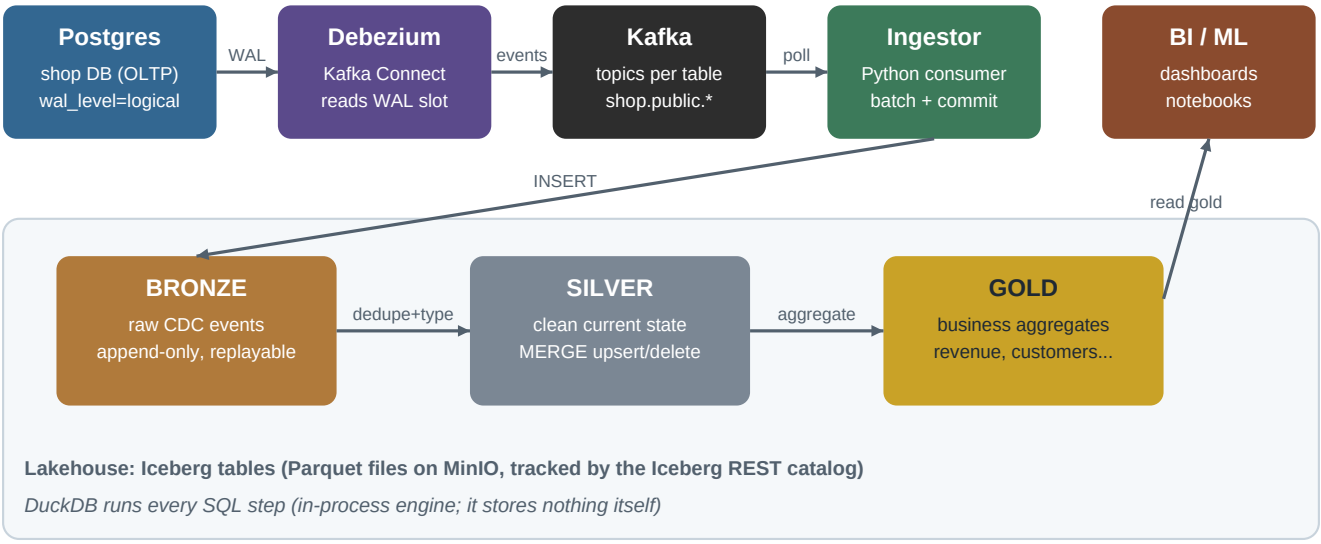
Memorize the shape of this answer. It's what you say when an interviewer asks "walk me through a project".

> "I built a local lakehouse that replicates an operational Postgres database into an analytics lake in near real time. Debezium reads Postgres's write-ahead log, a technique called **change data capture**, and publishes every insert, update and delete as an event to Kafka, one topic per table. A Python ingestor consumes those topics in batches and appends the raw events to **bronze** Apache Iceberg tables stored on S3-compatible object storage (MinIO). SQL models run in DuckDB, which deduplicates each key to its latest event and MERGEs it into **silver**, a clean, typed mirror of the source tables that also applies deletes. **Gold** tables aggregate silver into business metrics such as daily revenue, customer lifetime value and product sales. The pipeline is at-least-once, and the MERGE is idempotent, so replays never create duplicates. I verified that silver matches Postgres row for row with checksums. Because everything is Iceberg, the tables have ACID commits, snapshots and time travel, and any engine can read them, whether DuckDB, Spark, Trino or Snowflake."

Three ideas carry the whole design:

- **Capture changes from the log, not with queries.** CDC sees deletes and every intermediate state, and it barely loads the production database.
- **Separate raw history from cleaned state.** Bronze keeps everything, so silver and gold can always be rebuilt or fixed.
- **Separate storage from compute.** Data lives in open files on object storage, and the engine (DuckDB here) is replaceable.

2. Architecture at a glance



Component	Role in LakeRun	Why this choice	Production equivalents
Postgres 16	Source OLTP database (users, orders, order_items). `wal_level=logical` lets the WAL be decoded into row changes.	Most common app database, with built-in logical replication.	RDS/Aurora Postgres, MySQL, MongoDB
Debezium (Kafka Connect)	Reads the replication slot and turns each row change into a JSON event.	Standard open-source CDC; does the initial snapshot plus streaming.	Debezium on MSK Connect, Confluent, Fivetran, AWS DMS
Kafka (KRaft)	Durable, ordered, replayable log of events; one topic per table.	Decouples producers from consumers; replay is possible by resetting offsets.	Amazon MSK, Confluent Cloud, Redpanda
Ingestor (Python)	Consumes topics, batches events, writes bronze, then commits offsets.	Simple and transparent: you can see every step.	Kafka Connect Iceberg sink, Flink, Spark Structured Streaming
MinIO	S3-compatible object storage holding Parquet data and Iceberg metadata.	Real S3 API, locally.	Amazon S3, GCS, Azure Blob/ADLS
Iceberg REST catalog	Maps table name to its current metadata file; makes commits atomic.	Open standard catalog API usable by any engine.	AWS Glue, Snowflake Polaris/Open Catalog, Unity Catalog, Nessie
DuckDB 2.0	Runs all SQL: bronze inserts, silver MERGE, gold aggregates, ad-hoc queries.	In-process, fast, zero ops; reads and writes Iceberg.	Spark, Trino/Athena, Snowflake, Databricks SQL

3. Core concepts, from source to dashboard

OLTP vs OLAP

OLTP (online transaction processing) databases like Postgres serve the app: many small reads and writes by primary key, row-oriented storage, strict consistency. **OLAP** (online analytical processing) systems serve analysis: few, huge queries that scan millions of rows but only a few columns, so they use columnar storage. Running heavy analytics on the production database slows the app down, and that is the reason to copy data into a separate analytics store at all.

Change Data Capture (CDC)

CDC means capturing every change made to a database and shipping it elsewhere. There are two families:

	Query-based (polling)	Log-based (what LakeRun uses)
How	`SELECT * WHERE updated_at > :last_run` on a schedule	Read the database's transaction log (Postgres WAL)
Deletes	Invisible: the row is simply gone	Captured as explicit delete events
Intermediate states	Lost if a row changes twice between polls	Every change captured, in commit order
Load on source	Repeated table scans	Very low; reads the log
Needs	A reliable updated_at column	Logical replication enabled, plus a replication slot

Postgres WAL, replication slots and publications

- **WAL (write-ahead log)**: before Postgres changes a data page, it appends the change to the WAL. That makes it crash-safe, and it means the WAL is an ordered history of every change.
- **wal_level=logical**: makes the WAL carry enough detail to reconstruct row-level changes (set in docker-compose).
- **Publication** (`lakesync_pub`): which tables get published.
- **Replication slot** (`lakesync_slot`): a bookmark that remembers how far Debezium has read. Postgres *keeps WAL until the slot confirms it*, so nothing is lost while Debezium is down. The trade-off: an abandoned slot makes WAL grow until the disk fills, which is a classic production incident.
- **LSN (log sequence number)**: a position in the WAL. Stored in bronze as `source_lsn`.
- **REPLICA IDENTITY FULL** (set on orders): delete and update events carry the whole old row in `before`. With the default identity, only the primary key is included.

The Debezium event envelope

Every Kafka message value looks like this (trimmed). The message key is the primary key, `{"id": 1}`.

```
{
  "op": "u",                                  // c=create  u=update  d=delete  r=snapshot read
  "before": {"id":1,"user_id":1,"status":"pending","total":"499.00", ...},
  "after":  {"id":1,"user_id":1,"status":"paid",   "total":"499.00", ...},
  "source": {"lsn": 26758608, "ts_ms": 1793943422145, "table": "orders", "snapshot": "false", ...},
  "ts_ms": 1793943422254
}
```

- **Snapshot (op=r)**: on first start, Debezium reads the existing rows, then switches to streaming from the WAL.
- **Tombstone**: after a delete, Debezium also sends a message with a null value. That lets Kafka log compaction drop the key. The ingestor skips tombstones.
- **Decimal handling**: by default NUMERIC is sent as base64 bytes ("AMLs" = 499.00). LakeRun sets `decimal.handling.mode=string` and decodes old events with a SQL macro.

Kafka essentials

- **Topic**: a named, append-only log; here one per table (`shop.public.orders`).
- **Partition**: a topic is split into partitions for parallelism. **Order is guaranteed only within a partition.** Debezium keys messages by primary key, so all changes to one row land in the same partition, in order. Silver relies on that.
- **Offset**: the position of a message inside a partition. Together, (topic, partition, offset) uniquely identifies an event.
- **Consumer group**: consumers sharing a `group_id` split partitions between them and store their *committed offsets* in Kafka. LakeRun uses `lakerun-bronze`.
- **Retention and replay**: Kafka keeps messages for a configured period, so you can reset offsets and reprocess.
- **KRaft**: Kafka's built-in consensus, which replaces ZooKeeper.

Delivery semantics

Semantics	How it happens	Risk
At-most-once	Commit the offset, then process	A crash loses data
At-least-once (LakeRun)	Process (write to lake), then commit the offset	A crash between the two steps replays the batch, giving duplicates
Exactly-once (effective)	At-least-once plus an idempotent sink	None, as long as dedupe is correct

LakeRun achieves **effectively exactly-once results**: bronze may contain a duplicate after a crash, but silver dedupes by key and Kafka offset, and MERGE only applies newer events, so the outcome is the same.

Lakehouse, Iceberg and Parquet

A **data lake** is files on cheap object storage: scalable but historically without transactions, updates or schemas. A **warehouse** (Snowflake, BigQuery) has all of those, but it's a closed system. A **lakehouse** adds a **table format** on top of lake files to get warehouse features on open storage.

- **Parquet**: a columnar file format. Queries read only the columns they need, values compress well, and per-column min/max statistics let engines skip files.
- **Apache Iceberg**: a table format. A table is a tree of metadata: *catalog pointer* to a *metadata.json* (schema, partition spec, snapshot list), to a *manifest list* per snapshot, to *manifests*, to the *Parquet data files*.
- **ACID commits**: a writer writes new files, then atomically swaps the catalog pointer to a new metadata.json. Readers see either the old snapshot or the new one, never half a write.
- **Snapshots and time travel**: every commit creates a snapshot, so you can query any of them (`AT (VERSION => id)`).
- **Row-level changes**: UPDATE, DELETE and MERGE are done with delete files (merge-on-read) or by rewriting files (copy-on-write).
- **Schema evolution** by column ID (add, rename and drop safely) and **hidden partitioning** (for example `day(ts)`, without the user filtering on a partition column).
- **Catalog**: the service that stores "table name -> current metadata file" and arbitrates concurrent commits. This project uses the Iceberg REST catalog.

DuckDB

DuckDB is an **in-process** analytical database, like SQLite but for analytics. It's a library inside the Python process, with a vectorized, columnar, multi-threaded engine. Its iceberg extension can `ATTACH` a REST catalog and read and write Iceberg tables, including INSERT, UPDATE, DELETE and MERGE. In LakeRun, DuckDB holds **no data**. It's pure compute, and every connection is in-memory. The lake is the source of truth, so you could swap DuckDB for Spark or Trino without moving data.

Medallion architecture

Layer	Contains	Rules	Who uses it
Bronze	Raw data exactly as received, plus ingestion metadata	Append-only, never edited; schema-light	Data engineers (replay, debugging, audit)
Silver	Cleaned, typed, deduplicated, conformed entities	One row per business key; deletes applied; consistent types	Analysts, data scientists, other pipelines
Gold	Business-level aggregates and metrics	Modeled for a use case; stable definitions	Dashboards, executives, ML features, apps

4. Life of one order: the flow end to end

Follow a customer who places order #23 and then pays for it.

Step	What happens
1. App writes to Postgres	`INSERT INTO orders (user_id, status, total) VALUES (4, 'pending', 1299.50)`. The transaction commits, and the change is written to the WAL at some LSN.

Step	What happens
2. Debezium reads the WAL	Through the replication slot, Debezium decodes the change with the `pgoutput` plugin and builds an event: `op=c, before=null, after={id:23,...}`.
3. Kafka stores it	The event is produced to `shop.public.orders` with key `{"id":23}`. It lands at, say, partition 0, offset 41.
4. Ingestor buffers it	`consumer.poll()` returns the message. It is parsed into a bronze row (topic, partition, offset, op, before/after JSON, lsn, timestamps) and buffered.
5. Bronze commit	When the buffer reaches 1,000 rows, 10 seconds pass, or (in `--once` mode) the topics are drained, one DuckDB transaction INSERTs into all bronze tables. That is one atomic Iceberg commit. *Only then* are the Kafka offsets committed.
6. The customer pays	`UPDATE orders SET status='paid'` becomes a new event with `op=u` at offset 57, and bronze now has two rows for order 23.
7. Silver MERGE	`silver/orders.sql` reads only bronze rows above the watermark, parses the JSON, keeps the latest event per order_id (`QUALIFY row_number() ... ORDER BY source_lsn DESC = 1`), and MERGEs it. Order 23 matches with a newer offset, so it is UPDATEd to `status='paid'`.
8. Gold rebuild	`daily_revenue`, `customer_summary`, `product_sales` and `order_status_funnel` are recomputed from silver. Order 23 now counts as revenue.
9. Consumption	A dashboard or analyst queries `lake.gold.daily_revenue`. Iceberg snapshots let you ask what it looked like yesterday.

The silver MERGE, annotated

```
MERGE INTO lake.silver.orders AS t
USING (
  SELECT CAST(coalesce(after::JSON->>'id', before::JSON->>'id') AS BIGINT) AS order_id,  -- deletes only have 'be
         op, ..., dbz_decimal(after::JSON->>'total', 2) AS total, source_lsn
  FROM lake.bronze.orders
  WHERE {new_events}          -- only offsets above the watermark, up to the high mark, per partition
  QUALIFY row_number() OVER (PARTITION BY order_id ORDER BY source_lsn DESC) = 1   -- latest per key
) AS s
ON t.order_id = s.order_id
WHEN MATCHED AND s.op = 'd'  AND s.source_lsn >= t._source_lsn THEN DELETE      -- deleted at source
WHEN MATCHED AND s.op <> 'd' AND s.source_lsn >  t._source_lsn THEN UPDATE ... -- only NEWER changes
WHEN NOT MATCHED AND s.op <> 'd'                               THEN INSERT ...; -- new key
-- same transaction: advance lake.ops.watermarks to the high marks, then COMMIT
```

Incremental: transform.py reads lake.ops.watermarks (highest merged offset per Kafka partition), finds the current high mark, MERGEs exactly that range and advances the watermark *in the same transaction*. Each run costs O(new events), not O(history). **Ordering by LSN** (the Postgres WAL position, unique per change) instead of Kafka offset keeps it correct even when a key moves to another partition after a topic is repartitioned. Why this is correct and idempotent: running it twice changes nothing (the LSN guard stops a second update, and a deleted key whose latest event is 'd' is neither matched nor inserted). Duplicated bronze rows from at-least-once delivery collapse in `row_number()`. Re-running a no-op MERGE even creates no new Iceberg snapshot.

5. The medallion layers in this project

Table	Grain	Key columns
bronze.users / orders / order_items	one row per CDC event	kafka_topic, kafka_partition, kafka_offset, op, before, after (JSON text), source_lsn, source_ts, ingested_at
silver.users	one row per user	user_id, email, name, created_at
silver.orders	one row per order	order_id, user_id, status, total DECIMAL(12,2), updated_at
silver.order_items	one row per order line	order_item_id, order_id, sku, quantity, price, line_total
gold.daily_revenue	one row per day	orders, paid_orders, cancelled_orders, revenue, avg_order_value, active_customers
gold.customer_summary	one row per customer	total_orders, lifetime_value, first/last order, segment (vip / active / prospect)

Table	Grain	Key columns
gold.product_sales	one row per SKU	units_sold, revenue, orders, customers, avg_price (excludes cancelled and pending)
gold.order_status_funnel	one row per status	orders, order_value, pct_of_orders

Every silver table also carries lineage columns: `_source_lsn` (the WAL position of the change that produced the row), `_source_ts` (when it changed in Postgres) and `_merged_at` (when it reached silver). `now() - _source_ts` is your **data freshness** metric.

Business rule used in gold: **revenue** counts orders whose status is paid, shipped or delivered. Pending and cancelled orders are excluded.

6. Design decisions and trade-offs

Interviewers care less about what you built than why you built it that way, and what you would change at scale.

Decision	Why (here)	At scale
Keep bronze raw and append-only	Bronze can rebuild any downstream table after a bug fix, and it keeps the full change history (an audit trail). Cost: storage, plus parsing JSON on every silver run.	Partition bronze by ingestion date and expire old snapshots; consider typed bronze with a schema registry (Avro/Protobuf).
At-least-once plus an idempotent MERGE	Much simpler than distributed exactly-once transactions; correctness lives in one place (silver).	Same idea in production. Add a watermark so silver processes only new bronze rows.
Order by Postgres LSN, not Kafka offset or timestamp	LSN is unique per change and increases per row (verified on 1.2M events). Offsets aren't comparable across partitions, and keys move partitions when a topic is repartitioned; timestamps can tie.	Same. Debezium also exposes a sequence field for finer ordering.
Incremental silver with per-partition watermarks	Each run merges only (watermark, high mark] and advances the watermark in the same transaction: cost tracks new data, and a crash can't skip or double-apply.	Also partition bronze by ingestion day so the range scan prunes whole partitions.
Parallel ingestors with a serialized commit	Kafka assigns each partition to one worker; parsing runs in parallel. A lock serializes only the short Iceberg commit because concurrent DuckDB appends lost data in testing.	Use an engine with proper optimistic-concurrency retries (Flink, Spark, Kafka Connect Iceberg sink) or a distributed lock.
Gold rebuilt with DROP + CREATE	Gold is small, and a full rebuild means no drift. (DuckDB-Iceberg doesn't support CREATE OR REPLACE.)	Incremental aggregates by date partition, or materialized views; avoid the brief 'missing table' window.
Hard deletes in silver	Silver mirrors the source exactly, so it can be checked with checksums.	Often soft-delete (is_deleted, deleted_at) or an SCD Type 2 history table for analytics.
Batch micro-commits (1,000 rows or 10 s)	Each Iceberg commit creates files and a snapshot; tiny commits cause the small-files problem.	Tune batch size against latency; run compaction (rewrite_data_files) and expire_snapshots.
One DuckDB transaction across bronze tables	A batch with orders and items commits atomically, so the tables never disagree.	Same; most engines support multi-table commits through the REST catalog.
DuckDB as the engine	Zero infrastructure and very fast on one machine; great for learning and medium data.	Spark or Flink for big or streaming transforms, Trino or Snowflake for many concurrent BI users. The tables stay the same.
SQL files instead of Python transforms	Readable, reviewable, portable to dbt.	dbt with tests (unique, not_null, relationships) and documentation.

7. What was broken, and how it was fixed

Good material for the "tell me about a bug you debugged" question.

Problem	Root cause	Fix
Iceberg REST container exited at startup	The image runs as a non-root user and couldn't create its SQLite catalog file in the root-owned `/data` volume	`user: root` on the service
Writes to the lake would fail	The `lake` bucket was never created in MinIO	One-shot `minio-init` service runs `mc mb --ignore-existing local/lake`; iceberg-rest waits for it to finish
Credential mismatch	`.pyiceberg.yaml` used minioadmin, but MinIO uses admin / supersecurepassword123	Aligned credentials
Order totals arrived as "AMLs"	Debezium's default `decimal.handling.mode=precise` encodes NUMERIC as base64 bytes	Connector set to `string`; `dbz_decimal()` macro decodes the old events
Parallel writers silently lost data (found while scaling)	Concurrent DuckDB appends to the same Iceberg table: some commits reported success, but their snapshot was overwritten (4 of 39 commits lost in one test)	Commit lock around the write; parsing stays parallel. Verified 120/120 commits kept
Ingestor stored nothing, and only the first topic ever ran	It only logged polled messages; `await start()` looped forever inside a for-loop over topics; nothing committed offsets	One consumer subscribed to all topics, batching, an atomic lake write, then a manual offset commit

Verification: after two rounds of simulated inserts, updates and deletes, the row counts and md5 checksums of every silver table matched Postgres exactly, and re-running the transform was a no-op.

8. How this works in a real product

Picture an e-commerce company. The product team owns the app and its Postgres database, and the business wants dashboards, reports and ML that don't slow down checkout. That is exactly the problem LakeRun solves. Here is how each piece looks in production.

What the business gets

- **Near-real-time dashboards** (revenue today, conversion funnel, top products) built on gold tables, with minutes of latency instead of a nightly dump.
- **Zero load on production**: analysts never query the app database.
- **History and audit**: bronze shows every status change an order went through (for disputes, fraud and compliance), and time travel shows what a report said last Tuesday.
- **One copy, many engines**: BI on Trino or Snowflake, data science on Spark or DuckDB notebooks, all reading the same Iceberg tables.
- **Downstream uses**: ML features (customer LTV, churn signals), reverse ETL (push segments to CRM or email tools), search indexing, cache invalidation.

The production version of each piece

Concern	In LakeRun	In production
Orchestration	You run `main.py run` by hand	Airflow, Dagster or Prefect schedule the transforms; the ingest runs as an always-on service on Kubernetes or ECS
Ingestion	Python consumer	Kafka Connect Iceberg sink, Flink, or Spark Structured Streaming; horizontally scaled consumer groups
Transforms	SQL files	dbt models with tests, docs and lineage; CI runs them on every PR
Schemas	JSON, no registry	Avro or Protobuf with Schema Registry; compatibility rules stop breaking changes
Table layout	Unpartitioned	Partitioned (for example `day(updated_at)`), sorted, with scheduled compaction and snapshot expiry
Catalog and security	REST fixture, no auth	Glue, Polaris or Unity with OAuth, table and column grants, encryption, PII masking

Concern	In LakeRun	In production
Observability	Logs	Alerts on consumer lag, replication slot WAL size, data freshness (now() - _source_ts), row-count anomalies, failed DAG runs
Data quality	Checksum by hand	Automated reconciliation, dbt or Great Expectations tests, data contracts with source teams
Privacy	n/a	GDPR or right-to-be-forgotten deletes propagated from the source; expire snapshots so deleted data is really gone
Schema changes	Edit SQL	Iceberg schema evolution; Debezium forwards new columns; silver updated deliberately

Typical operational incidents (and what to say about them)

- **The replication slot grows without bound**: Debezium has been down for hours, so Postgres retains WAL and the disk fills. Mitigate with alerts on `pg_replication_slots` lag, `max_slot_wal_keep_size`, and heartbeats.
- **Consumer lag rising**: the ingestor can't keep up. Scale consumers (up to the partition count) and increase batch size.
- **Small files**: frequent commits produce thousands of tiny Parquet files and slow queries. Schedule compaction.
- **Breaking source schema change**: a renamed column turns into nulls in silver. Prevent with schema registry compatibility, contracts and tests.
- **Backfill or bug fix**: fix the SQL, then rebuild silver and gold from bronze. This is the payoff of keeping raw data.

9. Interview questions and answers

Q. Why CDC instead of a nightly SELECT * export?

Log-based CDC captures deletes and every intermediate change, adds minutes instead of a day of latency, and puts almost no load on the source because it reads the WAL rather than scanning tables. Nightly exports miss deletes, double the load, and can't show history.

Q. What is a replication slot, and what can go wrong?

A server-side bookmark of how far a logical replication consumer has read. Postgres retains WAL until the slot advances, so no change is lost while the consumer is down. If the consumer dies and nobody notices, WAL accumulates and can fill the disk. You must monitor slot lag and drop abandoned slots.

Q. How do you guarantee ordering of changes for the same row?

Debezium uses the primary key as the Kafka message key, so all events for a row hash to the same partition, and Kafka preserves order within a partition. Silver picks the event with the highest Postgres LSN per key, which stays correct even if a repartition moves the key to another partition. There is no global order across rows, and none is needed.

Q. Is your pipeline exactly-once?

Delivery is at-least-once: I commit Kafka offsets only after the Iceberg commit succeeds, so a crash can replay a batch. Results are effectively exactly-once because silver dedupes per key and the MERGE only applies an event whose offset is newer than the stored one, which makes it idempotent.

Q. How are deletes handled end to end?

Postgres DELETE becomes a Debezium event with op='d' and the key in 'before', followed by a tombstone that I skip. Bronze stores the delete event. In silver, MERGE ... WHEN MATCHED AND op='d' THEN DELETE removes the row. For analytics you could keep a soft-delete flag instead.

Q. Why bronze, silver and gold? Isn't that triple storage?

Each layer has a different contract. Bronze is the immutable, replayable truth for recovery and audit. Silver is the clean, reusable entity layer that many consumers share. Gold is use-case-specific metrics with stable definitions. Storage is cheap; being unable to recompute after a bug is expensive.

Q. What does Iceberg add over plain Parquet files in S3?

Atomic commits (readers never see partial writes), snapshots and time travel, row-level updates, deletes and MERGE, safe schema evolution, hidden partitioning, and file-level statistics for pruning. Plus an open catalog that lets many engines share the same tables safely.

Q. How does an Iceberg commit work?

The writer writes new data files and new manifests, then writes a new metadata.json and asks the catalog to swap the table pointer from the old metadata to the new one with compare-and-swap. If another writer committed first, the swap fails and the writer retries on top of the new state. That is optimistic concurrency.

Q. Copy-on-write vs merge-on-read?

Copy-on-write rewrites affected data files during an update: writes are slower, reads are fast. Merge-on-read writes small delete files and merges them at read time: writes are fast, reads are slower until compaction. CDC-heavy tables usually prefer merge-on-read plus regular compaction.

Q. Why DuckDB? Would you use it in production?

It's an in-process vectorized engine with nothing to operate, and fast on a single node. It's ideal here and in production for small to medium transforms, CI tests, and analyst notebooks. For terabyte-scale or highly concurrent workloads I'd use Spark, Flink or Trino on the same Iceberg tables. That's the benefit of separating storage from compute.

Q. What is the small-files problem?

Every commit adds files, so streaming small batches creates thousands of tiny files and a lot of metadata, and scan planning gets slow. Fix it by batching writes, scheduled compaction (rewrite_data_files), expiring snapshots, and sensible partitioning.

Q. How is silver incremental?

lake.ops.watermarks stores the highest merged Kafka offset per partition. Each run finds the current high mark, MERGEs exactly the range (watermark, high mark], and advances the watermark in the same transaction. Rows arriving mid-run wait for the next run. A no-op run over 2.6M bronze rows dropped from ~3 s (full scan) to 0.27 s.

Q. How did you scale the ingestor, and what went wrong?

I raised each topic to 6 partitions and ran 6 consumer processes in one group, with a rebalance listener that flushes and commits before giving partitions away. Testing showed that concurrent DuckDB appends to one Iceberg table can silently lose commits, so I parallelized parsing and serialized only the commit with a lock. Throughput went from ~4.1k to ~17k events/s, still matching the source row for row.

Q. A column is added to the orders table. What happens?

Debezium includes it in the 'after' JSON automatically. Bronze stores JSON text, so nothing breaks. Silver ignores it until I add the column with ALTER TABLE (Iceberg schema evolution is metadata-only) and update the MERGE. With a schema registry, compatibility rules would flag breaking changes.

Q. How do you know the lake is correct?

Reconciliation: compare row counts and checksums of silver against the source (I did this with md5 over ordered rows). Also data tests (unique keys, not-null, foreign keys) and freshness monitoring using now() - _source_ts.

Q. How would you scale this to 10,000 events per second?

Add partitions and scale ingestors in a consumer group (or use the Kafka Connect Iceberg sink or Flink), make silver incremental and partitioned, compact continuously, move transforms to Spark or Flink, add a schema registry, and orchestrate with Airflow or Dagster. Monitor consumer lag, slot lag and freshness.

Q. What is time travel useful for?

Reproducing a report as it was, debugging (diff two snapshots), recovering from a bad write by rolling back, and audit. In DuckDB: SELECT ... FROM lake.silver.orders AT (VERSION => snapshot_id).

Q. SCD Type 1 vs Type 2: which is your silver?

Silver is Type 1: it overwrites to the current state. Type 2 keeps every version with valid_from and valid_to. You could derive Type 2 straight from bronze, because bronze has every change with its timestamp.

10. Hands-on: commands and exercises

```
cd docker-compose && docker compose up -d && cd ..
curl -X POST -H 'Content-Type: application/json' \
     --data @docker-compose/connector.json localhost:8083/connectors   # first time only

pipenv run python main.py simulate     # inserts / updates / deletes in Postgres
pipenv run python main.py run          # Kafka -> bronze, then silver + gold
pipenv run python main.py status       # row counts per layer
pipenv run python main.py shell        # interactive SQL over lake.*
pipenv run python main.py ingest       # streaming mode (Ctrl+C to stop)
pipenv run python main.py ingest --workers 6 --batch-size 10000   # parallel consumers
pipenv run python main.py transform --full-refresh              # rebuild silver from bronze
pipenv run python scripts/load_test.py --rounds 10 --batch-size 1000   # 1M-event load test
```

Exercises that build real understanding

- **Watch CDC live**: open Kafka UI (localhost:8080), run `simulate`, and inspect messages in `shop.public.orders`. Find a u event and compare before with after.
- **See one order's full history**: `SELECT op, after::JSON->>'status', source_ts FROM lake.bronze.orders WHERE record_key = '{"id":5}' ORDER BY kafka_offset;`
- **Prove idempotency**: run `main.py transform` twice, then check `iceberg_snapshots(lake.silver.orders)`. No new snapshot appears.
- **Time travel**: take a snapshot_id, run simulate and run, then compare `count(*) ... AT (VERSION => id)` with the current count.
- **Look at the files**: MinIO console (localhost:9001) under lake/warehouse/, where metadata/*.json, manifest Avro files and data/*.parquet live.
- **Break it on purpose**: stop the ingestor halfway through a run, restart it, and confirm silver still matches Postgres. That is at-least-once in action.
- **Replay**: delete the consumer group's offsets (`kafka-consumer-groups.sh --reset-offsets --to-earliest`), re-ingest, and confirm silver is unchanged despite duplicate bronze rows.

11. Load test: 1 million events, before and after scaling

`scripts/load_test.py` writes change events straight into Postgres and times each stage of the pipeline separately. Every 100k-event round contains 10k user INSERTs, 30k order INSERTs, 40k order_item INSERTs and 20k order UPDATEs. Everything ran on one laptop in Docker, so treat these as local-dev numbers, not a cluster benchmark.

Run A: 1,000,000 events, default batch size (1,000 rows per Iceberg commit)

Stage	Result	What it tells you
Postgres writes	1M events in 13.9 s (~72k/s)	The source isn't the bottleneck.
Debezium -> Kafka	All 1M in Kafka 20.4 s after the first write (~49k/s); caught up 6.5 s after the last write	CDC keeps up comfortably. Replication slot lag peaked at 176 MB of retained WAL during the burst.
Kafka -> bronze (ingestor)	243 s, **~4,100 events/s**, 979 Iceberg commits, peak memory 178 MB	**The bottleneck.** One single-threaded Python consumer plus a commit roughly every 1,000 rows.
bronze -> silver (MERGE)	orders 1.5 s, items 1.1 s, users 2.2 s (full scan of ~1M bronze rows)	DuckDB is fast: the full-scan design is fine at this size.
silver -> gold	0.05 to 0.25 s per table	Aggregating silver costs almost nothing.
Re-run with no new data	0.25 s, no new snapshot	The MERGE is idempotent.
Bronze layout afterwards	1,500 files and 1,500 snapshots for 1M rows (~670 rows per file)	**The small-files problem**, produced by tiny commits.

Run B: 200,000 more events, batch size 50,000

Metric	Batch 1,000 (Run A)	Batch 50,000 (Run B)
Ingest throughput	~4,100 events/s	~7,700 events/s (1.9x)
Iceberg commits per 100k events	~98	~2
Peak ingestor memory	178 MB	258 MB
Worst-case freshness (time before an event is visible in bronze)	Low: under a second at load, 10 s flush timer	Higher: up to one 50k batch, still bounded by the 10 s timer

Run C: 1,000,000 events after the scaling work (6 partitions, 6 workers, batch 10,000, incremental silver)

Metric	Run A (before)	Run C (after)
Kafka -> bronze throughput	~4,100 events/s (243 s)	**~17,150 events/s (58 s), 4.2x**
Iceberg commits for 1M events	979	97
Silver: merge 1M new events	~4.8 s (full scan of bronze)	3.4 s (only the new range)
Silver re-run with no new data	~3.1 s at 1.2M bronze rows, growing with history	**0.27 s** at 2.6M bronze rows, flat
Correctness	checksums match	checksums match (220k users, 660k orders, 864k items)

The 58 s includes ~10 s of fixed overhead (process start, consumer-group rebalance, and the 6 s idle check before --once exits), so steady-state throughput is higher. Debezium again delivered 1M events in ~19 s. With ingest at ~17k/s, Debezium (~49k/s) is now only about 3x ahead, and the serialized commit is the next ceiling.

Correctness at scale: after both runs (1.2M events in total), the row counts and md5 checksums of silver.users (120,008), silver.orders (360,022) and silver.order_items (480,029) matched Postgres exactly. Gold and silver queries returned in 15 to 45 ms. Looking up one order's history in bronze took 37 ms despite the 1,500 small files.

How the system behaves under 1M traffic

- **It absorbs the burst; nothing is lost.** Kafka is the shock absorber: Debezium pushed all 1M events into Kafka within about 20 s, and the slower ingestor drained them over about 4 minutes. During that time, consumer lag (not data loss) was the symptom.
- **The bottleneck is the Python ingestor.** Run B shows that with commit overhead mostly gone, it's CPU-bound at roughly 8k events/s on JSON parsing and kafka-python decoding. Per-commit overhead was about 0.1 s (979 commits accounted for ~110 s of Run A's 243 s).
- **Sustained capacity** of one ingestor is ~4k to 8k events/s, which is 350M to 650M events per day. A sustained 1M events *per minute* (~17k/s) would make lag grow without bound. That's the signal to scale out.
- **The replication slot is the production database's risk.** While Debezium lags, Postgres retains WAL (176 MB here). With Debezium down for hours, that becomes gigabytes on the primary's disk.
- **Small files accumulate quietly.** Queries are still fast at 1.5k files, but planning cost grows with every commit. Compaction is required, not optional.

Scaling it: what to change, in order of payoff

Bottleneck	Fix	Expected effect
Commit overhead and small files	Bigger batches (10k to 50k rows) with a time-based flush; nightly compaction and snapshot expiry	~2x throughput, ~50x fewer files
Single-threaded Python	Faster client (confluent-kafka/librdkafka), orjson, or let DuckDB parse JSON in bulk	Several times faster per process
One partition per topic (DONE)	6 partitions and 6 ingestors in one consumer group (parallelism is capped at the partition count)	Measured 4.2x (Run C)
Custom ingestor at all	Kafka Connect Iceberg sink or Flink, which have commit coordination built in	Tens to hundreds of thousands of events/s
Silver full scan (DONE)	Watermark-based incremental MERGE; next, partition bronze by day	No-op run 3.1 s to 0.27 s (Run C)
WAL retention risk	Alert on slot lag and set max_slot_wal_keep_size	Protects the production database

Interview framing: *"I load-tested 1M CDC events end to end. Debezium sustained ~49k events/s and Kafka absorbed the burst. My single Python ingestor was the bottleneck at ~4k/s, and half of that time was Iceberg commit overhead, which also created 1,500 small files. Raising the batch size to 50k nearly doubled throughput and cut commits 50x. Silver still matched the source row for row. To scale further I'd partition topics and run parallel consumers, or move to the Iceberg Kafka Connect sink, and add compaction."*

12. Limitations and next steps

- The parallel ingestors share a local file lock for commits, so they must run on one machine (use a distributed lock or a different engine across machines).
- Bronze isn't partitioned, so the incremental range read relies on file statistics rather than partition pruning.
- Gold has a brief gap between DROP and CREATE. Next: transactional DELETE + INSERT, or incremental aggregates.
- No partitioning, compaction or snapshot expiry yet.
- `orders` has no created_at, so updated_at stands in for the order date in gold.
- JSON without a schema registry, and no automated data-quality tests. Next: dbt with tests, and Avro.
- The `dbz_decimal` base64 fallback ignores negative values (none exist here).
- No auth on the catalog or MinIO beyond static keys. Fine locally, never in production.